ELEC 4706
Lab3

A Short Explanation on Pipelining

Carleton University

Objectives

→ An introduction to the concept of pipelining

→ A hint for the Lab 3

Basic concept of pipelining

→ There are two basic techniques to increase the execution rate of a digital system:

- Increasing the clock rate,
- Increasing the number of operations that can be done simultaneously. General techniques in this regard are parallelism and pipelining.

→ Pipelining owes its origin to car assembly lines. The idea is to have more than one operation being processed by the hardware at the same time.

→ Similar to the assembly line, the success of a pipeline depends upon dividing the execution of an instruction among a number of subunits (stages), each performing part of the required operations.

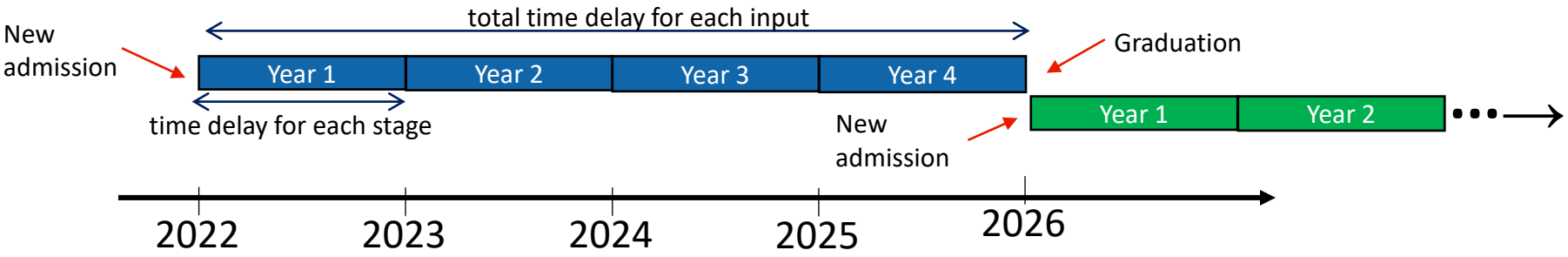
Modern Times (film), written and directed by Charlie Chaplin, 1936.

Basic concept of pipelining

→ Another example is your study at the Carleton University.

→ Imagine a group of students (Blue boxes) enter the university in 2022. They will be in the university for 4 years to be graduated. Two scenarios are possible for the university:

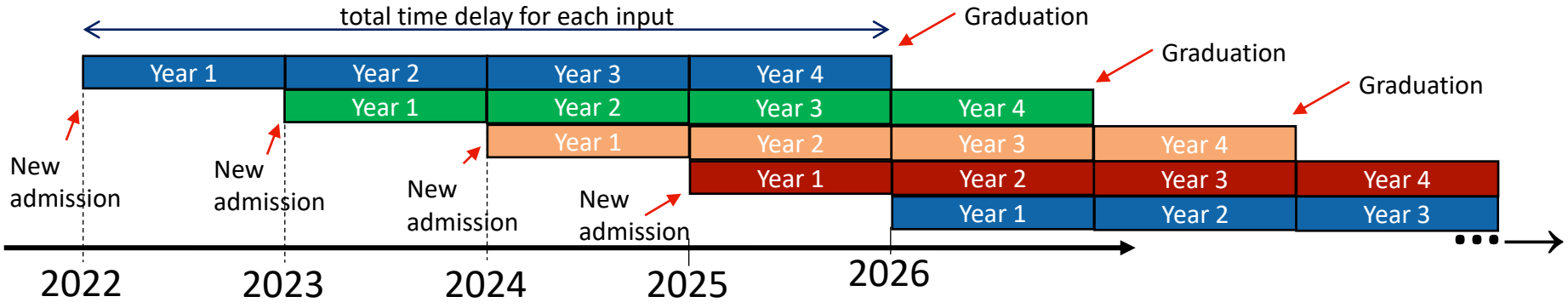
1. No pipelining: university accepts one group of students and spends all its resources for them till they graduate. After they graduated a new group of students will be admitted.



In this case, every four years one group of students will be graduated.

Basic concept of pipelining

2. Pipelined: University accepts a group of new students each year.



In this case, although each group spends four years at the university, but every year there will be a group of graduated students.

Basic concept of pipelining

→ Pipelining refers to the technique in which a given task is divided into a number of subtasks that need to be performed in sequence.

→ Each subtask is performed by a given functional unit. The units are connected in a serial fashion and all of them operate simultaneously.

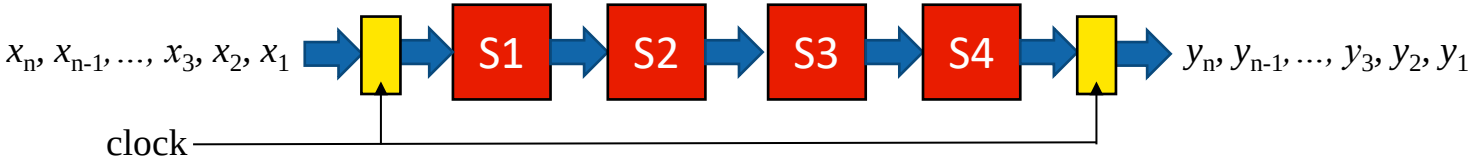
→ The use of pipelining improves the performance compared to the traditional sequential execution of tasks.

→ Our university education system shows an illustration of the basic difference between executing four subtasks of a given operation (in this case first year (Y1), second year (Y2), third year (Y3), and fourth year (Y4) of education) using pipelining and sequential processing.

→ **Note**: think about resources which are required in each scenario? What is the trade off?

Basic concept of pipelining

→ As another example, consider a task which has been implemented on a hardware (let say multiplication!). Let assume, somehow, we have been able to divide this task into four sub-tasks (stages): S1, S2, S3, S4. This hardware gets n inputs (x_1 to x_n) and produces n outputs (y_1 to y_n).



→ If the i^{th} sub-task (stage) has a t_{di} delay, the total delay time to process one data is

$$T_{clock} = t_{d1} + t_{d2} + t_{d3} + t_{d4}$$

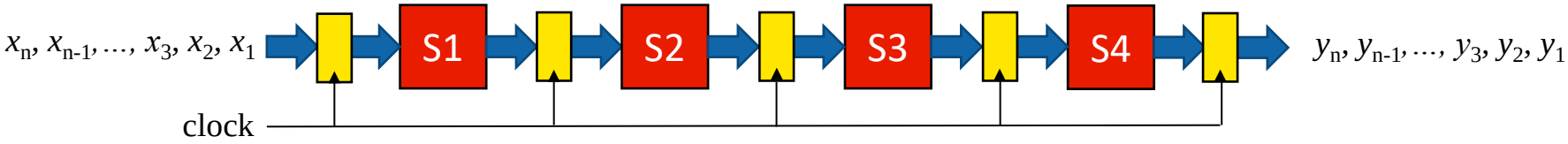
→ Therefore, since a new input can enter the hardware only when the previous one exists, to process n data (x_1 to x_n) the total delay will be

$$n \times (td_1 + t_{d2} + t_{d3} + t_{d4})$$

Basic concept of pipelining

→ Now we want to use pipelining technique to make this hardware faster.

1. Divide the task into m sub-tasks (in this example m=4 and stages are S1, S2, S3, S4) with equal (or almost equal) delays ($t_{d1} \approx t_{d2} \approx t_{d3} \approx t_{d4}$ in this example).
2. Separate stages from each other using clocked buffers. This will isolate stages and makes sure at each time step each data goes through one stage.
3. Choose system clock period equal to the maximum stage delay $T_{clock} = Max\{t_{d1}, t_{d2}, t_{d3}, t_{d4}\}$. In this way, we make sure all stages have enough time to perform their operations.



→ First data (x_1) needs $4 \times T_{clock} = 4 \times Max\{t_{d1}, t_{d2}, t_{d3}, t_{d4}\}$ time to g through the system and produce y_1. But every other output (y_2 to y_n) only need one clock to be processed.

→ Therefore, since we have m stages (here $m = 4$) to process n data (x_1 to x_n) the total delay will be

$$m \times T_{clock} + (n-1) \times T_{clock} = (m+n-1) \times T_{clock} = (m+n-1) \times Max\{t_{d1}, t_{d2}, t_{d3}, t_{d4}\}$$

Basic concept of pipelining

→ In general, assuming a hardware with n stages all of which have a delay of t, for n data inputs the pipelining speed-up, $S(n)$, is equal to

$$S(n) = \frac{m \times n \times t}{(m + n - 1) \times t} = \frac{m \times n}{(m + n - 1)}$$

→ Note:

$$\lim_{n \to \infty} S(n) = m$$

→ which means pipelining is more effective, when number of input data in high, and

$$\lim_{m \to \infty} S(n) = n$$

→ which means pipelining is more effective if you can divide your hardware into more small stages.

Basic concept of pipelining

→ For example: for a system with 4 stages, which each stage has a delay of 10 ns, pipelining speed-up for 1000 input data is

→ Delay without pipelining:

→ Time without pipelining $= 4 \times 1000 \times 10\,ns = 40000\,ns$

→ Time using pipelining $= (4 + 1000 - 1) \times 10\,ns = 10030\,ns$

$$S(1000) = \frac{4 \times 1000}{4 + 1000 - 1} = 3.98803589232$$

→ The pipelined system is almost 4 times faster!

Basic concept of pipelining

→ Question: can you discuss,

- What is the cost of this speed up?
- What if the delays of the stages are not close?

A Hint on Lab 3 Circuit

→ In lab 3, you are asked to design a pipelined (with two stages) an array multiplier that multiplies two 4-bit numbers.

→ Look at the longest path in this circuit and try to answer these questions:

- How many Full adders are on this path?
- What is the delay of the longest path (t_{LP}), in terms of t_{FA} (delay of one full adder)?
- Try to find out which final or intermediate output values are calculated just after $t_{LP}/2$?
- If a value can be calculated within $t_{LP}/2$, you can save that vale in a buffer (register) and release the unit for next input in the pipeline!

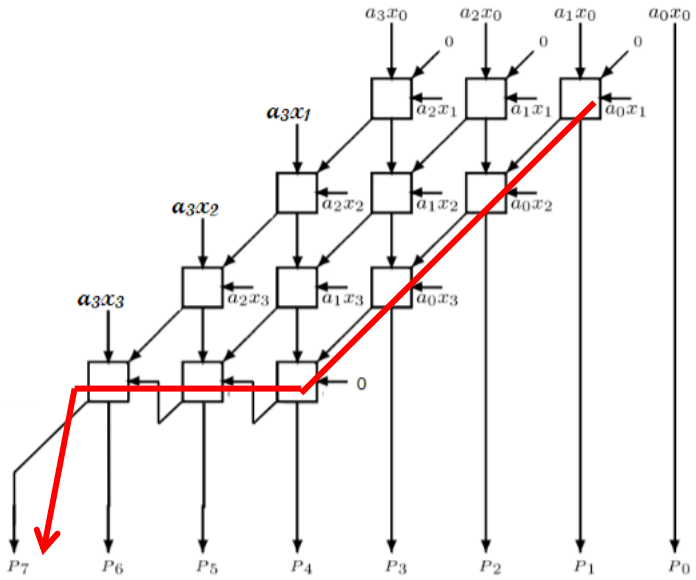


Figure 2. Architecture of array multiplier.